

AMBA AHB Lite Master

Implementation using Verilog HDL

Design Report

By Ahmed H. Ashour

Supported And Unsupported Features:

- Supports Basic Transfers
- Doesn't Support Locked Transfers (**HMASTLOCK** = 0)
- Doesn't Support Protected Transfers (**HPROT[3:0]** = 4'b0011) a non-cacheable, non-bufferable, privileged, data access.
- Only transfer of word size (32 bits) (**HSIZE[2:0]** = 3'b010)
- Only supports two burst types **HBURST[2:0]** = SINGLE (3'b000), INCR4(3'b011)
- Responds to slave errors **HRESP** by canceling the operation and returning to IDLE
- Supports Waited transfer by holding the data until the slave signals a ready using **HREADY**

Hardware Architecture: Consists of two FIFOs that hold the received and sent data and a FSM that controls the transfer of this data.

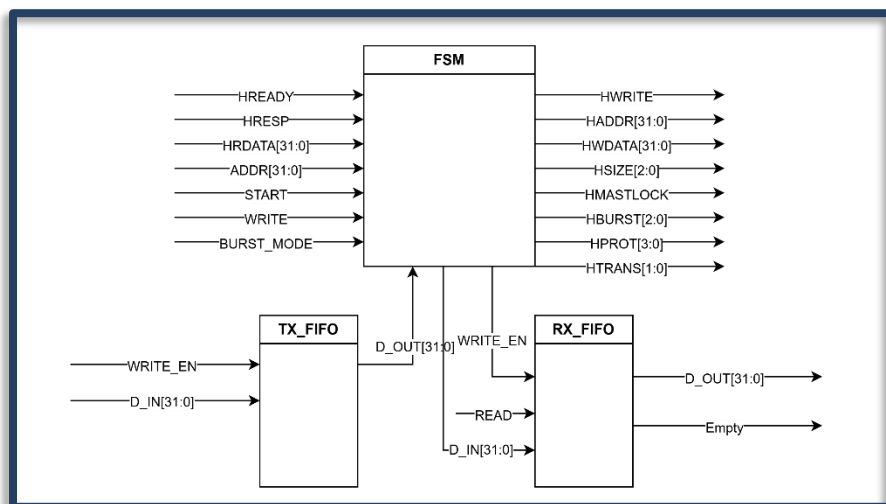


Figure 1 : AHV Master Block Diagram

Operation:

When the user triggers a transfer, the master latches the address, direction, and burst type, then drives the bus with NONSEQ followed by SEQ transfers for each beat. For writes, data is fetched from the transmit FIFO and placed on HWDATA; for reads, incoming HRDATA from the slave is stored into the receive FIFO. Each beat only advances when the slave asserts HREADY, and the master checks HRESP to determine if the transfer was successful. On an OKAY response, the burst continues incrementing the address each cycle on an ERROR response, both FIFOs are flushed by resetting them, and the master returns to idle.

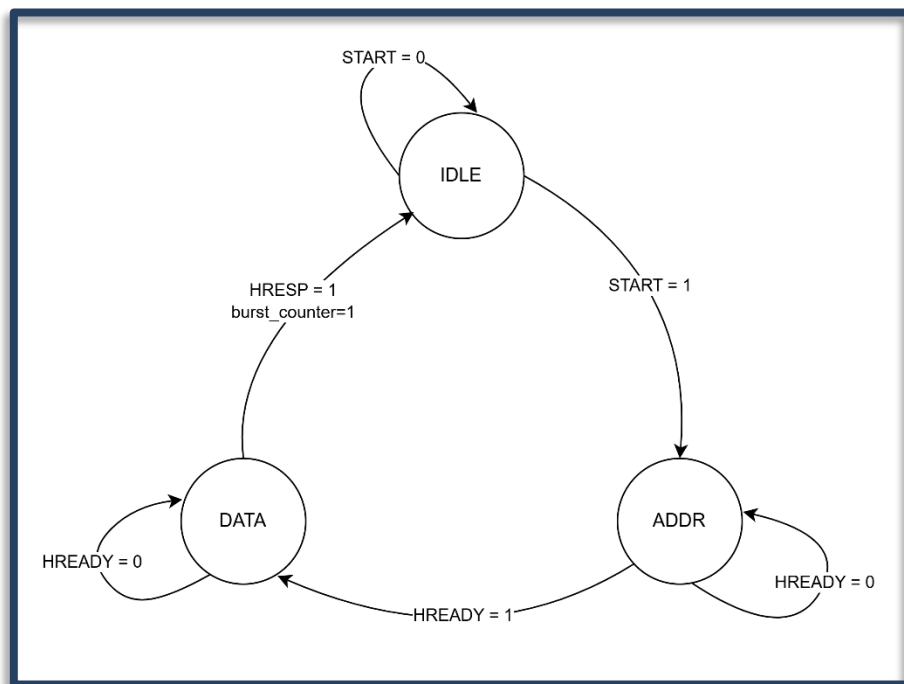


Figure 2 : FSM State Diagram

HDL Coding :

```
fifo.v

module fifo (
    input clk,
    input reset,
    input write_en,
    input read_en,
    input [31:0] din,          // fixed 32-bit input
    output [31:0] dout,       // fixed 32-bit output
    output full,
    output empty
);

    // Fixed depth = 4
    reg [31:0] memory [0:3]; // storage memory

    // Pointers sized for 4 entries
    reg [1:0] write_pointer = 0, read_pointer = 0; // 2 bits for 0..3
    reg [2:0] data_count = 0; // counts up to 4

    always @(posedge clk) begin
        if (!reset) begin
            write_pointer ≤ 0;
            read_pointer ≤ 0;
            data_count ≤ 0;
        end else begin
            if (write_en && !full) begin
                memory[write_pointer] ≤ din;
                write_pointer ≤ write_pointer + 1;
                data_count ≤ data_count + 1;
            end
            if (read_en && !empty) begin
                read_pointer ≤ read_pointer + 1;
                data_count ≤ data_count - 1;
            end
        end
    end

    // flag logic
    assign empty = (data_count == 0);
    assign full = (data_count == 4);

    // output logic
    assign dout = memory[read_pointer]; // fall-through read

endmodule
```

Figure 3 : FIFO Code

```

module ahb_lite_master (
    input HCLK, HRESETn,

    // user inputs
    input [31:0] user_addr,
    input user_write, // 1 = write and 0 = read
    input user_start, // trigger transaction
    input user_burst_mode, // 0 = single, 1 = incr4

    // tx fifo controls
    input user_fifo_write_en,
    input [31:0] user_fifo_din,

    // rx fifo controls
    input user_fifo_read_en, // user reads data from RX FIFO
    output [31:0] user_fifo_dout, // read data output
    output user_fifo_empty, // empty flag

    // slave response
    input HREADY, // slave ready
    input HRESP, // 0 = okay and 1 = error
    input [31:0] HRDATA, // read data

    // master outputs
    output reg [31:0] HADDR,
    output reg [31:0] HWDATA,
    output reg HWRITE,
    output reg [1:0] HTRANS,
    output reg [2:0] HSIZE,
    output reg [2:0] HBURST,
    output reg [3:0] HPROT,
    output reg HMASTLOCK;

);

//states
localparam IDLE = 2'b00, ADDR = 2'b01, DATA = 2'b10;
reg [1:0] state;

reg tx_fifo_rstn, rx_fifo_rstn;
reg [31:0] addr_reg;
reg write_reg;
reg [2:0] burst_count;

wire [31:0] tx_fifo_dout;
wire tx_fifo_full, tx_fifo_empty;
reg tx_fifo_read_en, rx_fifo_write_en;

fifo tx_fifo
(.clk(HCLK), .reset(tx_fifo_rstn), .write_en(user_fifo_write_en), .read_en(tx_f
.dout(tx_fifo_dout), .full(tx_fifo_full), .empty(tx_fifo_empty));

fifo rx_fifo
(.clk(HCLK), .reset(rx_fifo_rstn), .write_en(rx_fifo_write_en), .read_en(user_f
.dout(user_fifo_dout), .full(), .empty(user_fifo_empty));

always @(posedge HCLK or negedge HRESETn) begin
    tx_fifo_rstn ≤ HRESETn;
    rx_fifo_rstn ≤ HRESETn;
    if (!HRESETn) begin
        state ≤ IDLE;
        HTRANS ≤ 2'b00; // IDLE state
        HWRITE ≤ 1'b0;
        HADDR ≤ 32'b0;
        HWDATA ≤ 32'b0;
        HBURST ≤ 3'b000;
        burst_count ≤ 0;
        tx_fifo_read_en ≤ 0;
        rx_fifo_write_en ≤ 0;
    end else begin
        tx_fifo_read_en ≤ 0;
        rx_fifo_write_en ≤ 0;
        case (state)
            IDLE: if (user_start) begin
                // save user inputs
                addr_reg ≤ user_addr;
                write_reg ≤ user_write;

                HADDR ≤ user_addr;
                HWRITE ≤ user_write;
                HSIZE ≤ 3'b010; // word transfer
                HBURST ≤ user_burst_mode ? 3'b011 : 3'b000; // incr4 or single
                HPROT ≤ 4'b0011; // non cacheable 0 ,non bufferable 0, Privileged access 1 , data 1
                HMASTLOCK ≤ 1'b0; // no lock
                HTRANS ≤ 2'b10; // NONSEQ
                burst_count ≤ user_burst_mode ? 3'd4 : 3'd1;
                state ≤ ADDR;
            end

            ADDR: if (HREADY) begin
                if (write_reg && !tx_fifo_empty) begin
                    tx_fifo_read_en ≤ 1; // next word
                    HWDATA ≤ tx_fifo_dout; // drive data bus
                end
                state ≤ DATA;
            end

            DATA: if (HREADY) begin
                // check slave response
                if (HRESP == 1'b0) begin
                    // okay response
                    if (!write_reg) begin
                        // save read data into RX FIFO
                        rx_fifo_write_en ≤ 1;
                    end
                    burst_count ≤ burst_count - 1;
                    if (burst_count > 1) begin
                        // Continue burst
                        addr_reg ≤ addr_reg + 4; // increment by word size
                        HADDR ≤ addr_reg + 4;
                        HTRANS ≤ 2'b11; // SEQ
                        state ≤ ADDR;
                    end else begin
                        // end of burst
                        HTRANS ≤ 2'b00; // IDLE
                        state ≤ IDLE;
                    end
                end else if (HRESP == 1'b1) begin
                    // error response deletes data from both fifos
                    tx_fifo_rstn ≤ 1'b0;
                    rx_fifo_rstn ≤ 1'b0;
                    HTRANS ≤ 2'b00; // go to idle
                    state ≤ IDLE;
                end
            end
        endcase
    end
end
endmodule

```

Figure 4 : AHB Master Code

Testing

```

`timescale 1ns/1ps

module tb_v
    master tb_v

`timescale 1ns/1ps

module th_ahb_lite_master;

    reg HCLK, HRESETn, user_write, user_start, user_burst_mode;
    reg user_fifo_write_en, user_fifo_read_en, HREADY, HRESP;
    reg [31:0] user_addr, user_fifo_din, HWDATA;

    wire [31:0] user_fifo_dout, HADDR, HWDATA;
    wire user_fifo_empty, HWRITE, HMASTLOCK;
    wire [1:0] HTRANS;
    wire [2:0] HSIZE, HBURST;
    wire [3:0] HPROT;

    ahb_lite_master dut
    (.HCLK(HCLK), .HRESETn(HRESETn), .user_addr(user_addr), .user_write(user_write), .user_start(user_start),
    .user_burst_mode(user_burst_mode), .user_fifo_write_en(user_fifo_write_en), .user_fifo_din(user_fifo_din), .user
    .fifo_read_en(user_fifo_read_en),
    .user_fifo_dout(user_fifo_dout), .user_fifo_empty(user_fifo_empty), .HREADY(HREADY), .HRESP(HRESP), .HWDATA(HWDAT
    A),
    .HADDR(HADDR), .HWDATA(HWDATA), .HWRITE(HWRITE), .HTRANS(HTRANS), .HSIZE(HSIZE), .HBURST(HBURST), .HPROT(HPROT), .HMA
    STLOCK(HMASTLOCK));

    // clock
    initial begin
        HCLK = 0;
        forever #5 HCLK = ~HCLK;
    end

    // reset
    initial begin
        HRESETn = 0;
        #20 HRESETn = 1;
    end

    // slave tasks
    task slave_okay_response(input [31:0] rdata);
        begin
            HREADY = 1;
            HRESP = 0;
            HWDATA = rdata;
        end
    endtask

    task slave_error_response;
        begin
            HREADY = 1;
            HRESP = 1;
        end
    endtask

    task slave_wait_state;
        begin
            HREADY = 0;
        end
    endtask

    initial begin
        user_addr = 32'h8000_1000;
        user_write = 1;
        user_start = 0;
        user_burst_mode = 0;
        user_fifo_write_en = 1; // incrl
        user_fifo_read_en = 0;
        user_fifo_din = 0;
        user_fifo_read_en = 0;
        HREADY = 1;
        HRESP = 0;
        HWDATA = 32'h0;
    end

    // wait for reset
    @(posedge HRESETn);

    // load tx fifo with 4 words
    repeat (4) begin
        @(posedge HCLK);
        user_fifo_write_en = 1;
        user_fifo_din = $random;
    end

    @(posedge HCLK);
    user_fifo_write_en = 0;

    // start a write burst
    @(posedge HCLK);
    user_start = 1;
    @(posedge HCLK);
    user_start = 0;

    // slave responds okay each beat
    repeat (4) begin
        @(posedge HCLK);
        slave_okay_response(32'h6767_6767);
    end

    // wait for bus to be idle
    wait (HTRANS == 2'b00);

    // now test a read burst
    @(posedge HCLK);
    user_addr = 32'h8000_2000;
    user_write = 0;
    user_burst_mode = 1;
    user_start = 1;
    @(posedge HCLK);
    user_start = 0;

    // slave returns incrementing data
    repeat (4) begin
        @(posedge HCLK);
        slave_okay_response(32'h6767_6767);
    end

    // pop data from rx fifo
    repeat (4) begin
        @(posedge HCLK);
        if (!user_fifo_empty) begin
            user_fifo_read_en = 1;
            @(posedge HCLK);
            $display("read data from rx fifo: %h", user_fifo_dout);
            user_fifo_read_en = 0;
        end
    end

    // test error response
    @(posedge HCLK);
    user_addr = 32'h8000_3000;
    user_write = 1;
    user_burst_mode = 0;
    user_start = 1;
    @(posedge HCLK);
    user_start = 0;
    @(posedge HCLK);
    slave_error_response; #100 $stop;
end

endmodule

```

Figure 5 : Testbench Code

Results:

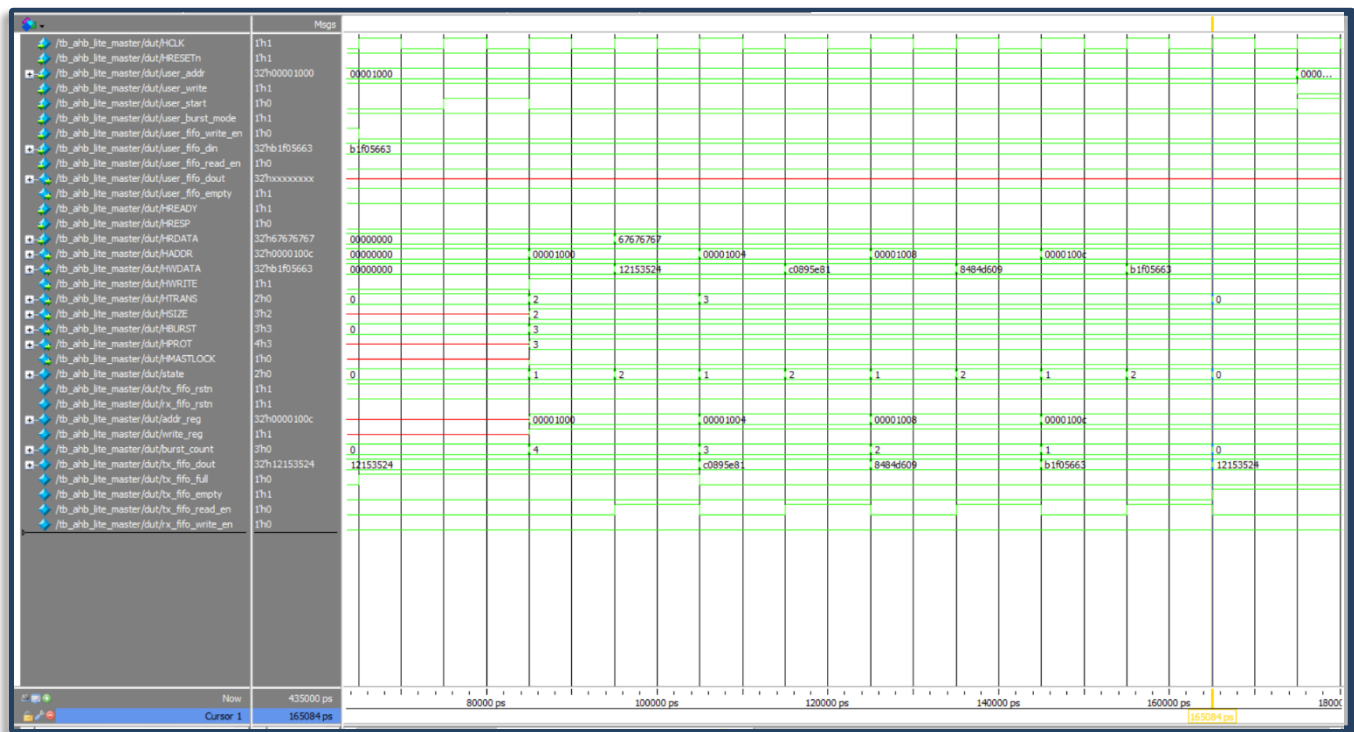


Figure 6 : Writing 4 beats with INCR4 mode

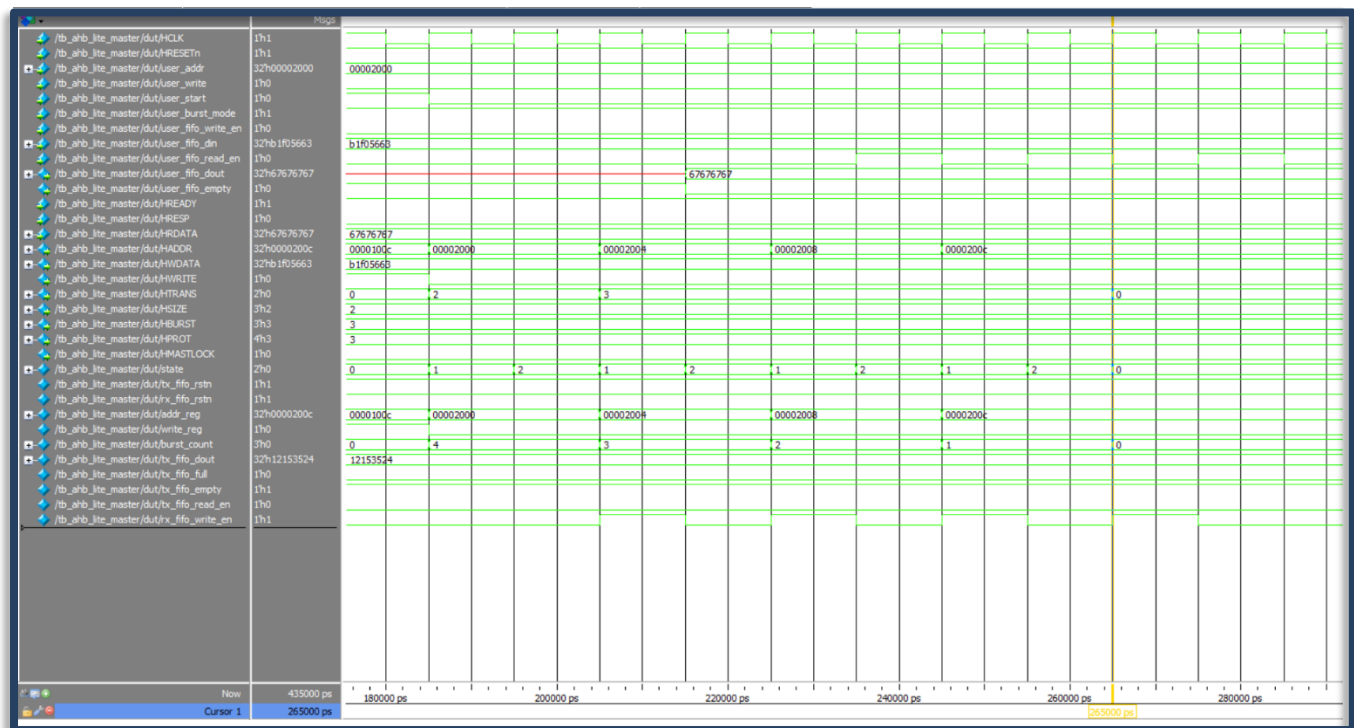


Figure 7: Reading 4 beats with INCR4 mode

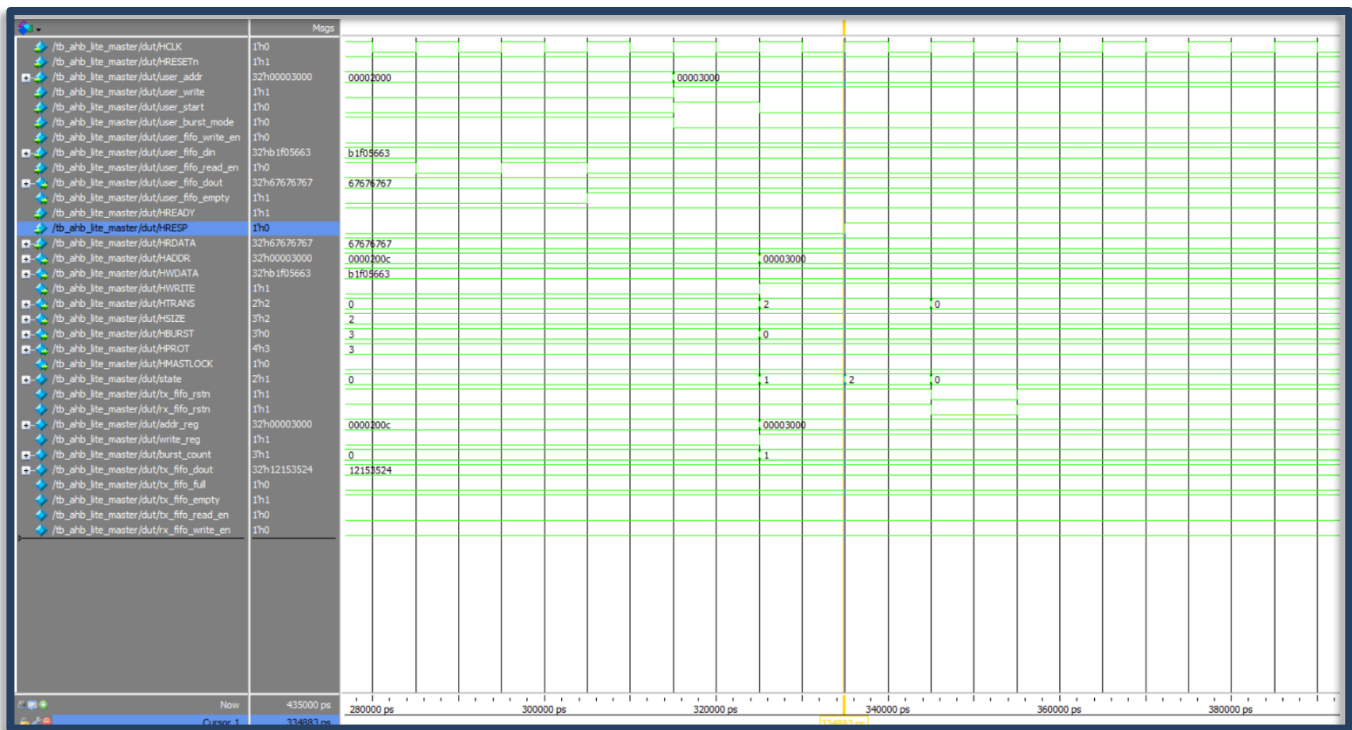


Figure 8 : Testing the Error response